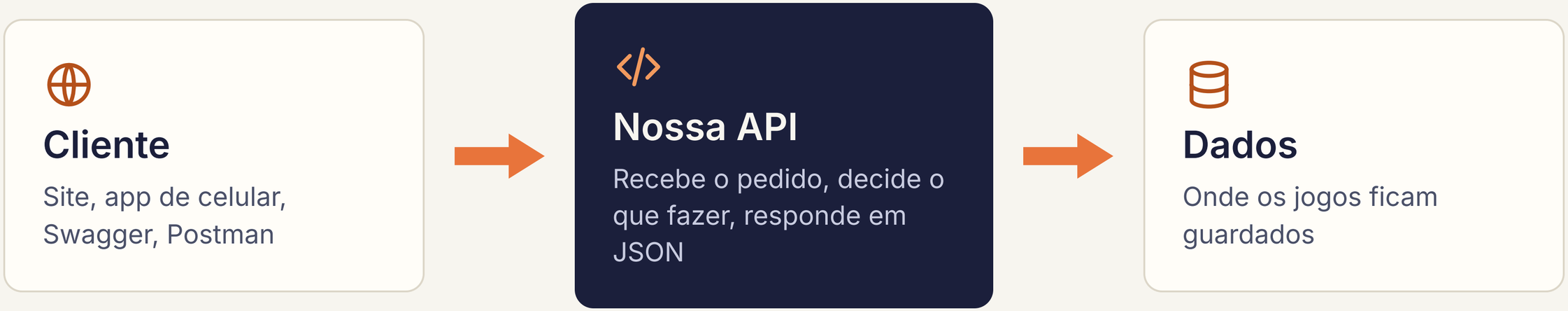
AULA 01 · ASP.NET CORE · .NET 10

Construindo uma API REST de Jogos

Versão 1: os dados vivem em uma lista na memória.
Depois, vamos levá-los para um banco com Entity Framework.

A IDEIA

Um catálogo de jogos que qualquer app pode consultar



Pense na API como o **balconista de uma loja de games**: você não entra no estoque, você pede no balcão.

CONCEITO

Todo pedido REST tem três partes

1

Recurso

A “coisa” que a API gerencia. Aqui: **Jogo**.

2

Endereço (URL)

Onde o recurso mora: **/api/Jogos** ou **/api/Jogos/1**.

3

Ação (verbo)

O que fazer com ele: **GET**, **POST**, **PUT**, **DELETE**.

```
GET /api/Jogos/1 → “me mostre o jogo de Id 1”
```

CONCEITO

Os verbos HTTP são o CRUD da API

Verbo	Rota	CRUD	Na prática
GET	/api/Jogos	Read	Listar todos os jogos
GET	/api/Jogos/{id}	Read	Buscar um jogo
POST	/api/Jogos	Create	Cadastrar um jogo novo
PUT	/api/Jogos/{id}	Update	Alterar um jogo existente
DELETE	/api/Jogos/{id}	Delete	Remover um jogo

CONCEITO

A resposta sempre vem com um código de status

200

OK

Deu certo, aqui está o dado

201

Created

Jogo cadastrado (POST)

204

No Content

Feito, nada a devolver (PUT, DELETE)

400

Bad Request

Erro do cliente: dado inválido

404

Not Found

Erro do cliente: Id não existe

Regra de bolso: **2xx = sucesso** · **4xx = o cliente pediu errado** · 5xx = a API falhou

O RECURSO

Um Jogo em C# vira um JSON na resposta

Models/Jogo.cs

```
public class Jogo
{
  public int Id { get; set; }
  [Required]
  public string Titulo { get; set; }
  public string Genero { get; set; }
  public string Plataforma { get; set; }
  public int AnoLancamento { get; set; }
  public bool Zerado { get; set; }
}
```



Resposta de GET /api/Jogos/1

```
{
  "id": 1,
  "titulo": "Zelda: TotK",
  "genero": "Aventura",
  "plataforma": "Switch",
  "anoLancamento": 2023,
  "zerado": true
}
```

ATENÇÃO: VERSÃO 1

Por enquanto, não há banco de dados.
Os jogos vivem em uma lista na memória.

Como um quadro branco: funciona durante a aula, mas é apagado quando a aplicação reinicia. Na versão 2, trocamos o quadro por um caderno: o **Entity Framework** com um banco de verdade.

VERSÃO 1 · LISTA EM MEMÓRIA

A “base de dados” é uma lista dentro do controller

```
private static readonly List<Jogo> _jogos =
[
  new Jogo { Id = 1, Titulo = "Zelda: TotK", … },
  new Jogo { Id = 2, Titulo = "Silksong", … },
  new Jogo { Id = 3, Titulo = "EA FC 26", … }
];

// quem gera o Id somos nós (por enquanto)
private static int _proximoId = 4;
```

Por que static?

O ASP.NET cria um controller novo a cada requisição. Sem static, a lista nasceria vazia em todo pedido.

Já começa com 3 jogos

Assim o primeiro GET já devolve algo para ver.

VERSÃO 1 · LISTA EM MEMÓRIA

Anatomia de um endpoint: buscar por Id

```
[Route("api/[controller]")]
[ApiController]
public class JogosController : ControllerBase

[HttpGet("{id}")]
public ActionResult<Jogo> GetJogo(int id)
{
  var jogo = _jogos.FirstOrDefault(j => j.Id == id);
  if (jogo is null) return NotFound();
  return Ok(jogo);
}
```

[Route]

[controller] vira “Jogos”: o nome da classe sem “Controller”.

[HttpGet("{id}")]

Liga o verbo GET e o pedaço {id} da URL ao parâmetro id.

NotFound() / Ok()

Viram os status 404 e 200.

VERSÃO 1 · LISTA EM MEMÓRIA

Cadastrar: o POST e o status 201

```
[HttpPost]
public ActionResult<Jogo> PostJogo(Jogo jogo)
{
  jogo.Id = _proximoId++;
  _jogos.Add(jogo);

  return CreatedAtAction(
    nameof(GetJogo),
    new { id = jogo.Id }, jogo);
}
```

O JSON vira objeto

O corpo da requisição é convertido no parâmetro jogo sozinho.

Nós geramos o Id

Na versão 2, quem gera é o banco.

CreatedAtAction

Responde 201 e diz no cabeçalho Location onde achar o jogo novo.

VERSÃO 1 · LISTA EM MEMÓRIA

Validação de graça com [ApiController]

No modelo

```
[Required]
public string Titulo { get; set; }

[Range(1970, 2100)]
public int AnoLancamento { get; set; }
```



POST com título vazio

```
400 Bad Request
{
  "errors": {
    "Titulo": ["O título é
      obrigatório."]
  } …
```

Repare: o método **PostJogo** nem chega a rodar. O ASP.NET barra o pedido antes e já responde 400.

OS LIMITES DA VERSÃO 1

A lista funciona na aula, mas não em produção

Reiniciou, perdeu

Memória é volátil: todo cadastro some quando a API para.

Cada servidor tem a sua

Com duas cópias da API rodando, cada uma vê uma lista diferente.

Id na mão

O _proximold reinicia em 4 e pode repetir com acessos simultâneos.

Tudo carregado sempre

Com milhões de jogos, a lista não cabe na memória nem é rápida de buscar.

PRÓXIMA AULA · VERSÃO 2

Entity Framework: um tradutor entre C# e o banco



A grande notícia: **o cliente não percebe a troca**. Rotas, verbos, JSON e status continuam iguais. Só a caixa “Dados” muda por dentro.

PREPARANDO A VERSÃO 2

O que muda e o que fica

Parte	Versão 1 · lista	Versão 2 · Entity Framework
Onde ficam os dados	List<Jogo> static	Tabela Jogos no banco (via DbContext)
Quem gera o Id	_proximoId++	O próprio banco
Buscar por Id	_jogos.FirstOrDefault(...)	await _context.Jogos.FindAsync(id)
Salvar	_jogos.Add(jogo)	Add(jogo) + SaveChangesAsync()
Métodos	Síncronos	async / await
Modelo Jogo	Igual	Igual
Rotas, verbos, status	Iguais	Iguais

RECAPITULANDO

Os passos que fizemos até aqui

1

Atualizar

net8.0 → net10.0 no .csproj

2

Simplificar

Tirar EF, DbContext, migrations e connection string

3

Modelar

Criar a classe Jogo com validações

4

Expor

JogosController com a lista e os 5 endpoints

5

Testar

Swagger e o arquivo .http

Conceitos que levamos daqui

recurso · rota · verbo HTTP · código de status · JSON · controller · validação · ciclo de vida (static)

MÃO NA MASSA

Rodando e testando a API

1. Subir a API

```
cd APIAula01
dotnet run
```

2. Swagger

Abrir **localhost:5135/swagger** e usar “Try it out” em cada endpoint.

3. Arquivo .http

No VS Code ou Visual Studio, clicar em “Send Request” em cada bloco.

Roteiro sugerido

Listar → cadastrar um jogo → listar de novo → buscar o Id 99 (404) → cadastrar sem título (400) → reiniciar a API e listar (sumiu!)

EXERCÍCIO

Sua vez: evolua a API de Jogos

1. Crie **GET /api/Jogos/plataforma/{nome}** que lista só os jogos daquela plataforma.
2. Adicione a propriedade **Nota** (0 a 10) com validação de faixa.
3. Desafio: impeça cadastrar dois jogos com o mesmo título (responda 409 Conflict).

Guarde o código: na próxima aula ele ganha um banco de dados de verdade.